DriveSure

Technologies Used in the Car Insurance Company Project

This PDF summarizes the frontend, backend, database, reporting, security, deployment, and development technologies used in the DriveSure project.

Technology	Used For	Project Relevance
Project Name	DriveSure	Full-stack car insurance company management system.
Domain	Car Insurance	Customer, vehicle, policy, claim, payment, report, and security workflows.
Architecture	MERN Stack	React frontend, Express backend, MongoDB database, and Node.js runtime.
Repository Evidence	package.json, README, routes, models, utilities	Technology list is based on files present in the project workspace.

1. Technology Stack Overview

DriveSure uses a MERN-style architecture with React on the client side, Node.js and Express.js on the server side, and MongoDB Atlas as the database. The application also includes utilities for authentication, upload handling, reporting, email notifications, security auditing, pagination, PDF generation, invoice generation, and CSV export.

Technology	Used For	Project Relevance
React 18	Frontend user interface	Builds dashboard screens, forms, protected routes, customer portal, and module pages.
Vite	Frontend build tool	Provides fast development server and optimized frontend build output.
Tailwind CSS	Styling framework	Supports responsive dashboard UI, layout utilities, spacing, colors, and component styling.
Node.js	Backend runtime	Runs the server-side JavaScript application.
Express.js	Backend API framework	Defines REST routes for auth, customers, vehicles, policies, claims, payments, reports, security, and notifications.
MongoDB Atlas	Cloud database	Stores insurance records using flexible document collections.
Mongoose	ODM layer	Defines schemas, validation rules, ObjectId relationships, and model operations.

2. Frontend Technologies

The frontend is located in the frontend folder and uses React with Vite. The route structure in src/App.jsx connects public pages such as login, register, reset password, email verification, customer login, and customer portal with protected application modules such as dashboard, customers, vehicles,

policies, claims, payments, reports, security center, notifications, backups, and profile.

Technology	Used For	Project Relevance
React	Component-based UI	Used for pages, shared components, dashboard cards, upload controls, timeline views, forms, and protected route handling.
React DOM	Browser rendering	Mounts the React application into the web page.
React Router DOM	Client-side routing	Supports login, portfolio, customer portal, and protected dashboard navigation.
Axios	HTTP client	Connects frontend pages to backend REST APIs.
Lucide React	Icon library	Provides clean dashboard and action icons.
Tailwind CSS	Utility-first styling	Creates responsive layouts and modern interface styling.
Vercel Analytics	Frontend analytics	Supports usage and performance visibility when deployed.
Vercel Speed Insights	Performance monitoring	Helps monitor frontend speed and user experience.

Frontend package evidence

Package	Version
@vercel/analytics	^2.0.1
@vercel/speed-insights	^2.0.0
@vitejs/plugin-react	^4.3.4
axios	^1.7.9
lucide-react	^0.468.0

Package	Version
react	^18.3.1
react-dom	^18.3.1
react-router-dom	^7.1.1
@eslint/js (dev)	^10.0.1
autoprefixer (dev)	^10.4.20
eslint (dev)	^10.0.0
eslint-plugin-react-hooks (dev)	^7.1.1
eslint-plugin-react-refresh (dev)	^0.5.2
globals (dev)	^17.6.0
postcss (dev)	^8.4.49
tailwindcss (dev)	^3.4.17
vite (dev)	^6.0.7

3. Backend Technologies

The backend is located in the backend folder and is organized into config, controllers, middleware, models, routes, services, utilities, uploads, reports, seed.js, and server.js. The API layer uses Express routes and controller functions to manage insurance operations.

Technology	Used For	Project Relevance
Node.js	Server runtime	Executes the backend API server and utility scripts.
Express.js	REST API framework	Creates API endpoints for authentication, dashboard, customer, vehicle, policy, claim, payment, report, and security modules.
express-async-handler	Async route handling	Simplifies error handling in asynchronous controller functions.
CORS	Cross-origin access	Allows configured frontend origins to communicate with the backend API.
Morgan	HTTP logging	Logs backend requests for development and debugging.
dotenv	Environment config	Loads secrets, URLs, SMTP settings, database connection string, and security keys.
Nodemon	Development restart tool	Restarts the server automatically during local backend development.

Backend package evidence

Package	Version
@vercel/speed-insights	^2.0.0
bcryptjs	^2.4.3
cors	^2.8.6
dotenv	^16.4.7

Package	Version
express	^4.21.2
express-async-handler	^1.2.0
jsonwebtoken	^9.0.2
jspdf	^4.2.1
jspdf-autotable	^5.0.8
mongoose	^8.24.0
morgan	^1.10.0
multer	^2.1.1
nodemailer	^8.0.7
pdfkit	^0.16.0
nodemon (dev)	^3.1.9

4. Database and Data Modeling

MongoDB Atlas is used as the target database service. Mongoose models represent business collections such as User, Customer, Vehicle, Policy, Claim, Payment, PaymentPlan, Notification, LoginHistory, SecurityAlert, ActivityLog, SupportTicket, BackupSnapshot, Garage, Feedback, and MessageThread.

Technology	Used For	Project Relevance
MongoDB Atlas	Cloud database hosting	Stores application records with cloud accessibility and deployment compatibility.
Mongoose	Schema and relationship modeling	Defines ObjectId references, validation, enums, defaults, timestamps, and model methods.
Seed script	Development data setup	Creates sample data for testing and demonstration.
Relational SQL reference	database/insurance.sql	Provides an additional relational schema reference for academic documentation.

5. Authentication and Cyber Security Technologies

Technology	Used For	Project Relevance
JWT	Token-based authentication	Protects API sessions and allows authenticated frontend requests.
bcryptjs	Password hashing	Hashes user passwords before storage and verifies login credentials securely.
Role-based access control	Authorization design	Restricts actions by roles such as admin, manager, agent, surveyor, and customer.
fieldCrypto utility	Sensitive field encryption	Supports encryption for sensitive data such as contact fields.

Technology	Used For	Project Relevance
LoginHistory model	Login monitoring	Records login attempts and supports suspicious activity review.
SecurityAlert model	Security dashboard	Stores and displays security-related alerts.
riskScoring utility	Fraud analysis	Assists claim risk scoring using fraud indicators.

6. Reporting, Files, and Communication

Technology	Used For	Project Relevance
PDFKit	Server-side PDF generation	Creates policy certificates, invoices, and operational PDF outputs.
jsPDF	PDF creation utilities	Supports browser/server report generation scenarios.
jsPDF AutoTable	PDF tables	Formats tabular report data inside PDFs.
CSV exporter utility	CSV download	Exports structured customer, policy, claim, payment, and report data.
Multer	File upload handling	Accepts claim evidence and document uploads.
Nodemailer	Email notifications	Supports verification emails, password resets, and alert communication.
Brevo API variables	Deployment email fallback	README documents Brevo variables for production transactional email.

7. Deployment and DevOps Technologies

Technology	Used For	Project Relevance
Render	Backend deployment	render.yaml and README deployment variables support hosting the Express API.
Vercel	Frontend deployment	frontend/vercel.json supports frontend deployment configuration.
Netlify	Alternative frontend deployment	frontend/netlify.toml and public redirects support static hosting.

Technology	Used For	Project Relevance
Postman	API testing	postman_collection.json documents and tests API workflows.
ESLint	Code quality	Checks frontend JavaScript and React code quality.
Git	Version control	Tracks source files and documentation changes in the repository.

8. Module-to-Technology Mapping

Technology	Used For	Project Relevance
Authentication	React forms, Express routes, JWT, bcryptjs, Nodemailer	Registration, login, reset password, verification, and secure sessions.
Customer Management	React pages, Axios, Express controllers, Customer model	Create, update, view, search, and manage customer details.
Vehicle Management	Vehicle page, vehicle routes, Vehicle model	Vehicle registration and customer-vehicle association.
Policy Management	Policy page, PDFKit, Mongoose Policy model	Policy creation, renewal, status tracking, and certificates.
Claims Management	Multer, Claim model, riskScoring, upload middleware	Claim submission, evidence upload, review, inspection, fraud scoring, and settlement.
Payment Management	Payment model, invoiceGenerator, report utilities	Premium collection, payment tracking, invoice generation, and payment reports.
Security Center	LoginHistory, SecurityAlert, ActivityLog, security middleware	Failed login review, audit logs, suspicious activity, and alerts.
Reports Dashboard	PDFKit, jsPDF, AutoTable, CSV exporter	Downloadable reports for business and academic review.

Conclusion

The DriveSure project uses a practical and modern technology stack suitable for a B.Tech full-stack and cyber security project. The stack combines React-based user interfaces, Express REST APIs, MongoDB data modeling, secure authentication, upload handling, reporting tools, and deployment-ready configuration. Together, these technologies support a complete car insurance company management system.